



Distributed Join Processing

Shimin Luo and Estella Xu

Part I: Join Strategies



Lecture Outline

Part I:

- Background
- Distributed Join Strategies
- Skew-Resistant Joins



Lecture Outline

Part I:

- Background
- Distributed Join Strategies
- Skew-Resistant Joins



Review

Considerations in Traditional DBs:

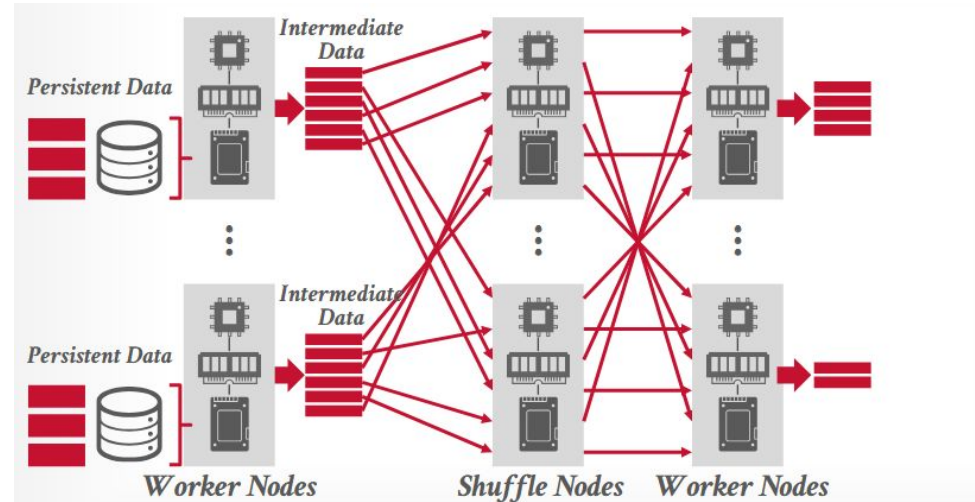
- Join Types: the semantics of the join
→ *e.g. cross join, semi join*
- Join Algorithms: how the join is implemented → *e.g. nested-loop, sort-merge*
- Join Tree Shapes: how to do multiple joins → *e.g. left-deep*

Additional Consideration for Distributed DBs:

- Join Graph Topology: how partitions of data are processed. Depends on...
 - Partitioning Schemes → *what partition functions, partition keys, etc. are used to partition the data across nodes?*
 - Data Exchange Operators → *how do we modify the partitioning scheme to our advantage?*
 - Distribution Policies → *do we duplicate when distributing?*

Review, cont.

In this lecture, we'll refer to the strategies of re-partitioning / broadcasting data as “join strategies” (not to be confused with “join algorithms”, which are what execute the join in individual nodes).





An Example

```
SELECT *  
FROM Customers  
JOIN Orders  
ON Customers.CustomerID = Orders.CustomerID;
```

Table R: Customers

CustomerID	Name	Address
1	Alice	Chicago
2	Bob	Seattle
3	Charlie	New York
...	...	...

Table S: Orders

OrderID	CustomerID	Item
101	1	Apple
102	2	Banana
103	1	Orange
104	2	Pear
...	...	...
500	9	Peach



Recall

CustomerID	Name	Address
1	Alice	Chicago
2	Bob	Seattle
3	Charlie	New York

OrderID	CustomerID	Item
101	1	Apple
102	2	Banana
103	1	Orange
104	2	Pear

Inner Join: $R \bowtie S$

CustomerID	Name	Address	OrderID	Item
1	Alice	Chicago	101	Apple
1	Alice	Chicago	103	Orange
2	Bob	Seattle	102	Banana
2	Bob	Seattle	104	Pear



Recall

CustomerID	Name	Address
1	Alice	Chicago
2	Bob	Seattle
3	Charlie	New York

OrderID	CustomerID	Item
101	1	Apple
102	2	Banana
103	1	Orange
104	2	Pear

Left Join (a.k.a. Left Outer Join): $R \bowtie S$

CustomerID	Name	Address	OrderID	Item
1	Alice	Chicago	101	Apple
1	Alice	Chicago	103	Orange
2	Bob	Seattle	102	Banana
2	Bob	Seattle	104	Pear
3	Charlie	New York	NULL	NULL



Lecture Outline

Part I:

- Background
- Distributed Join Strategies
- Skew-Resistant Joins



Observation

The primary challenges and considerations are the following:

- Network overhead and data partitioning costs
- Data locality
- Data replication
- Load scheduling



Observation

The primary challenges and considerations are the following:

- Network overhead and data partitioning costs
- Data locality
- Data replication
- Load scheduling

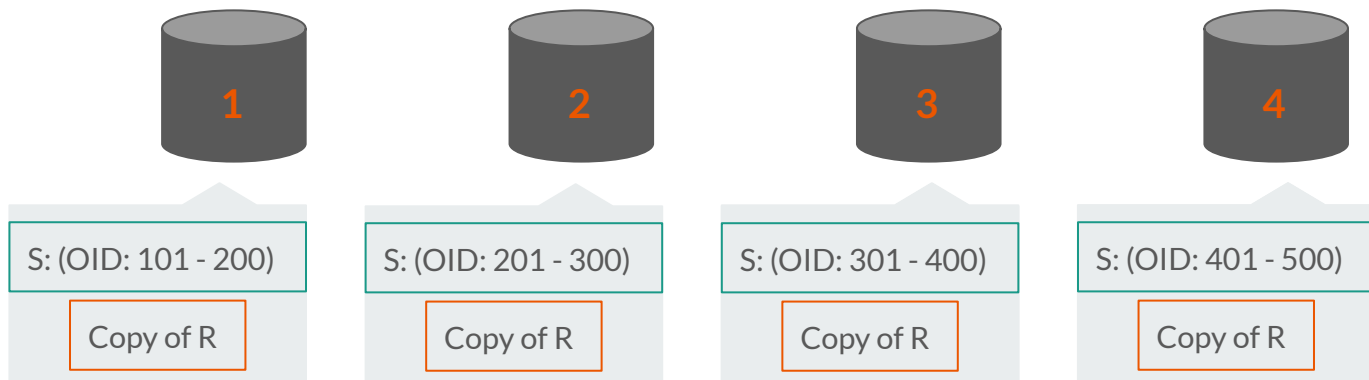
R: Customers (Smaller)
S: Orders (OID stands for OrderID)

Broadcast Join

Partition the larger table (typically under one key, called the partition key).

Replicate the smaller table and send a copy to each node.

Each node joins its local data in parallel and sends results to a coordinator.





Broadcast Join Observations

- Useful when one data set is significantly smaller than the other and fits into the memory of each node.
- Inefficient when both tables are large.

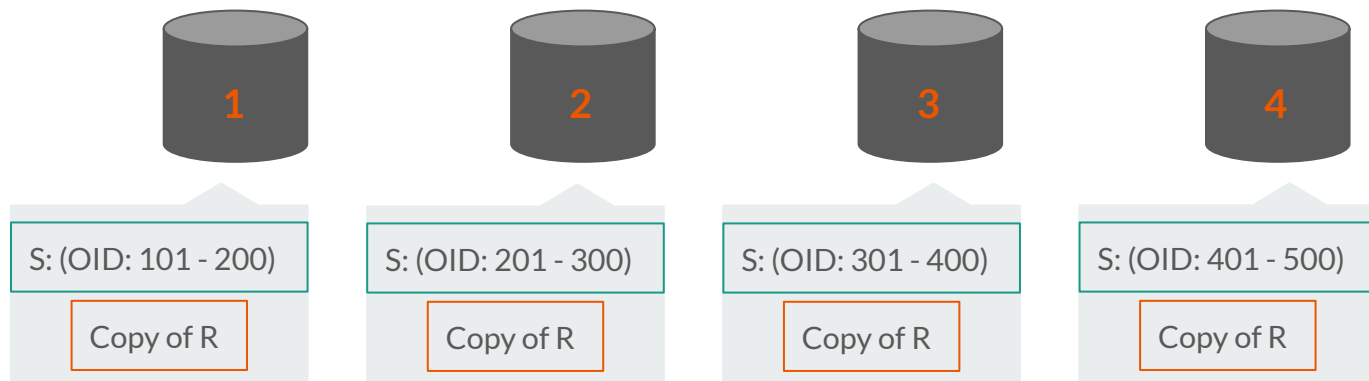
We'll talk more about this later.

A Problem

Suppose Charlie makes only one order and we attempt to left join at each node and join the results together.

CustomerID	Name	Address
1	Alice	Chicago
2	Bob	Seattle
3	Charlie	New York

OrderID	CustomerID	Item
101	1	Apple
102	2	Banana
103	1	Orange
104	2	Pear
...	...	...
400	3	Peach
...	...	...



CID	Name	OID	Item
1	Alice	101	Apple
2	Bob	102	Banana
1 & 2	A & B	103 - 200	...
3	Charlie	NULL	NULL

CID	Name	OID	Item
1	Alice	301	Peach
2	Bob	302	Plum
1 & 2	A & B	303 - 399	...
3	Charlie	400	Peach

...

...



S: (OID: 101 - 200)

S: (OID: 201 - 300)

S: (OID: 301 - 400)

S: (OID: 401 - 500)

Copy of R

Copy of R

Copy of R

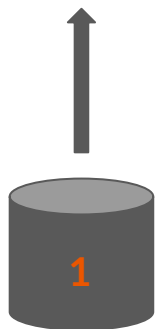
Copy of R

CID	Name	OID	Item
1	Alice	101	Apple
2	Bob	102	Banana
1 & 2	A & B	103 - 200	...
3	Charlie	NULL	NULL

CID	Name	OID	Item
1	Alice	301	Peach
2	Bob	302	Plum
1 & 2	A & B	303 - 399	...
3	Charlie	400	Peach

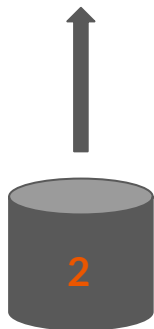
CID	OID	Item
1 & 2	101 - 500	...
3	NULL	NULL
3	NULL	NULL
3	400	Peach
3	NULL	NULL

Incorrect rows with
NULL values!



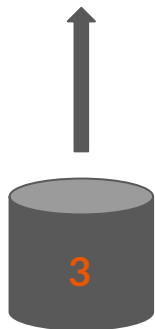
S: (OID: 101 - 200)

Copy of R



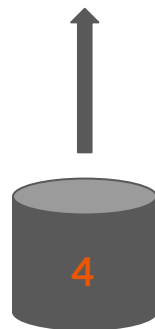
S: (OID: 201 - 300)

Copy of R



S: (OID: 301 - 400)

Copy of R



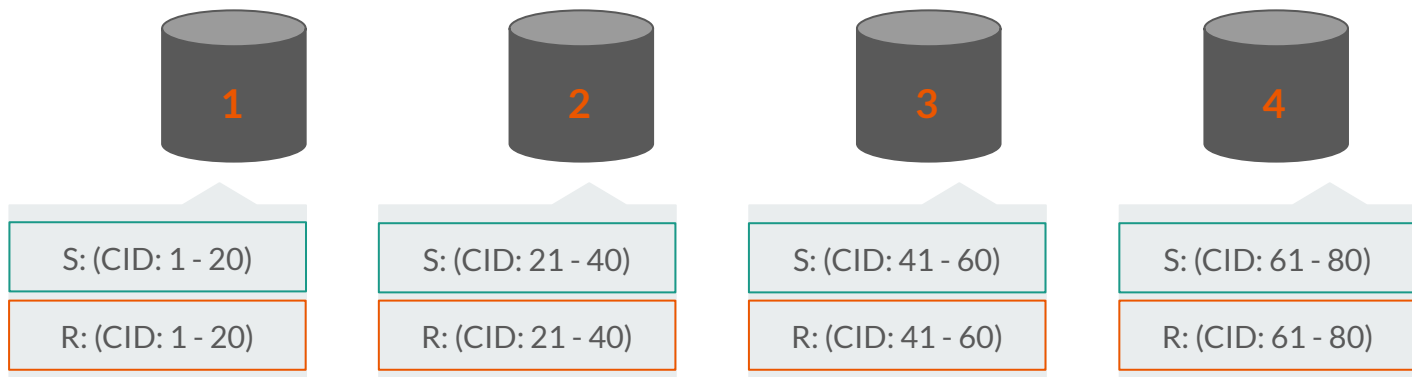
S: (OID: 401 - 500)

Copy of R

Shuffle Join

Shuffle both tables across the nodes based on the join key.

All rows with the same join key are located on the same node.





Shuffle Join Observations

- Useful when both datasets are large and do not fit in each node's memory.
- Has much more overhead.



High-Frequency Key Values

Definition. A high-frequency key value is a value of a specific key that occurs many times.

- In this example, the skewed key is CustomerID.
- The high-frequency key value is 2.

OrderID	CustomerID	Item
101	1	Apple
102	2	Banana
103	1	Orange
104	2	Pear
105	2	Apricot
106	2	Avocado
107	2	Cherry
108	2	Mango
...	...	...



Why might this be an issue in distributed joins?

Consider the distributed shuffle or hash join method.

Suppose we attempt a distributed hash join when customer 2 creates a large amount of orders. We hash each key value to a separate node. For example, if we have 4 nodes, we can have a hash function that sends CustomerID c to node $c \% 4$. Then, all of Customer 2's orders are hashed and sent to the same node, creating massive workload in that singular node.

Solution: skew-resistant join strategies.



Lecture Outline

Part I:

- Background
- Distributed Join Strategies
- Skew-Resistant Joins



The Skew-Resistant Join Process

1. Identify what the high-frequency key values are.
2. Transform the inputs in some way, i.e., prepare the join inputs.
3. Perform any non-distributed join operation in each join node.

We'll look at three specific strategies for step 2 (for inner join specifically).

- Broadcast Skew Join
- Full Skew Join
- Hybrid Skew Join



Broadcast Skew Join

1. Identify the skewed join keys.
 - Let S be the set of high-frequency (skewed) join key values.
 - Let N be the remaining (normal) join key values.
2. Perform a regular distributed join strategy on the values in N .
3. For each value in S , acquire all rows from the smaller table with that value.
Broadcast this subset of the smaller table to each node.
4. Union the results.

Initial Tables

Customers

CID	Name	Address
1	Alice	Chicago
2	Bob	Seattle
3	Charlie	New York
4	Dave	LA
5	Ed	Portland
6-26	...	...

Orders

OID	CID	Item
101 - 180	1	...
181 - 280	5	...
281 - 350	...	...

} 80 orders for Alice

} 100 orders for Ed

} all other orders

Step 1: Split into the small tables.

C_norm

CID	Name	Address
2	Bob	Seattle
3	Charlie	NY
4	Dave	LA
6 - 26	...	...

C_skew

CID	Name	Address
1	Alice	Chicago
5	Ed	Portland

O_norm

OID	CID	Item
281 - 350	...	...

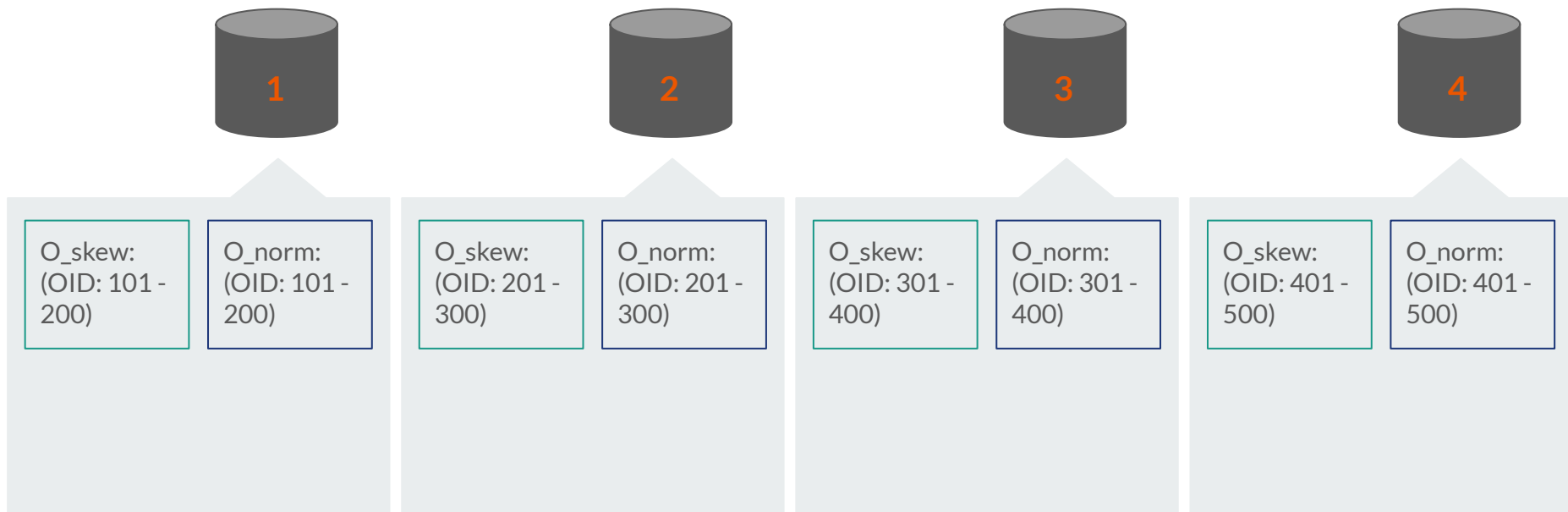
} all other orders

O_skew

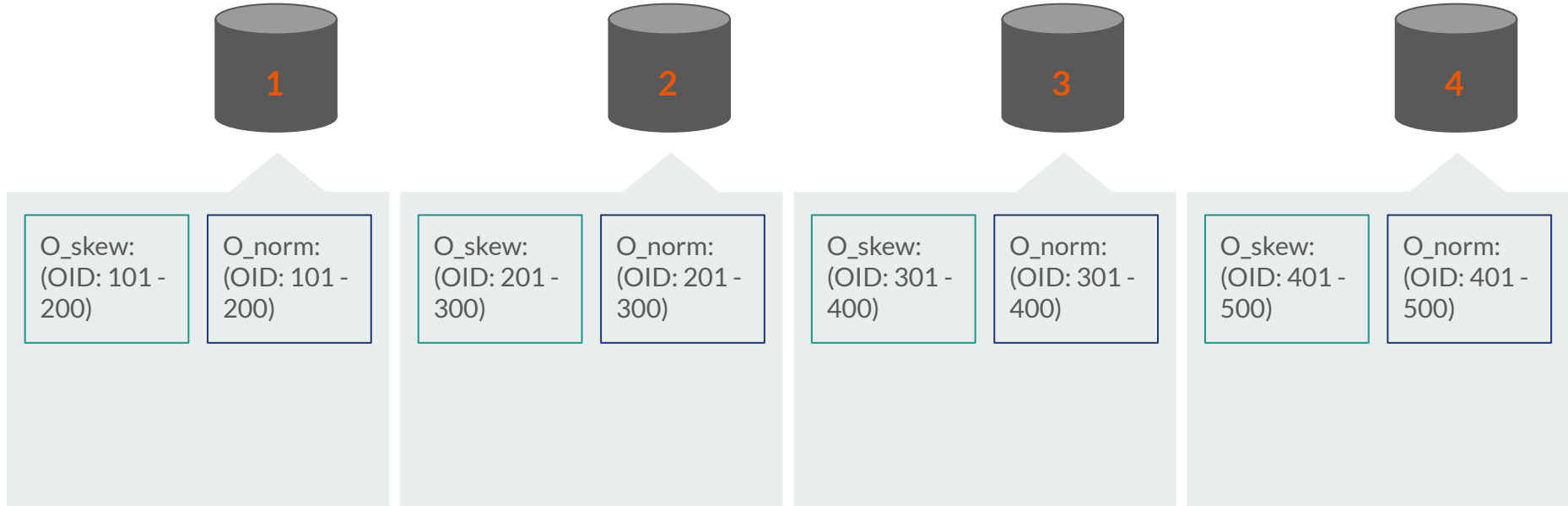
OID	CID	Item
101 - 180	1	...
181 - 280	5	...

} 80 orders for Alice
} 100 orders for Ed

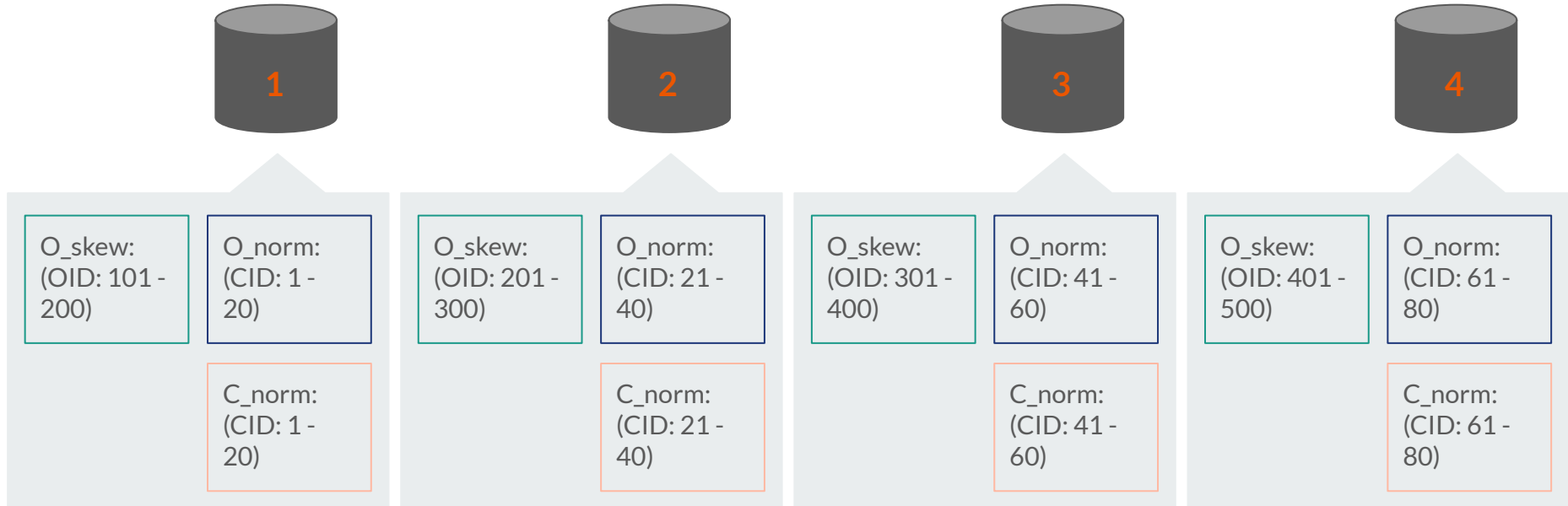
Suppose the Orders table is initially partitioned like so.



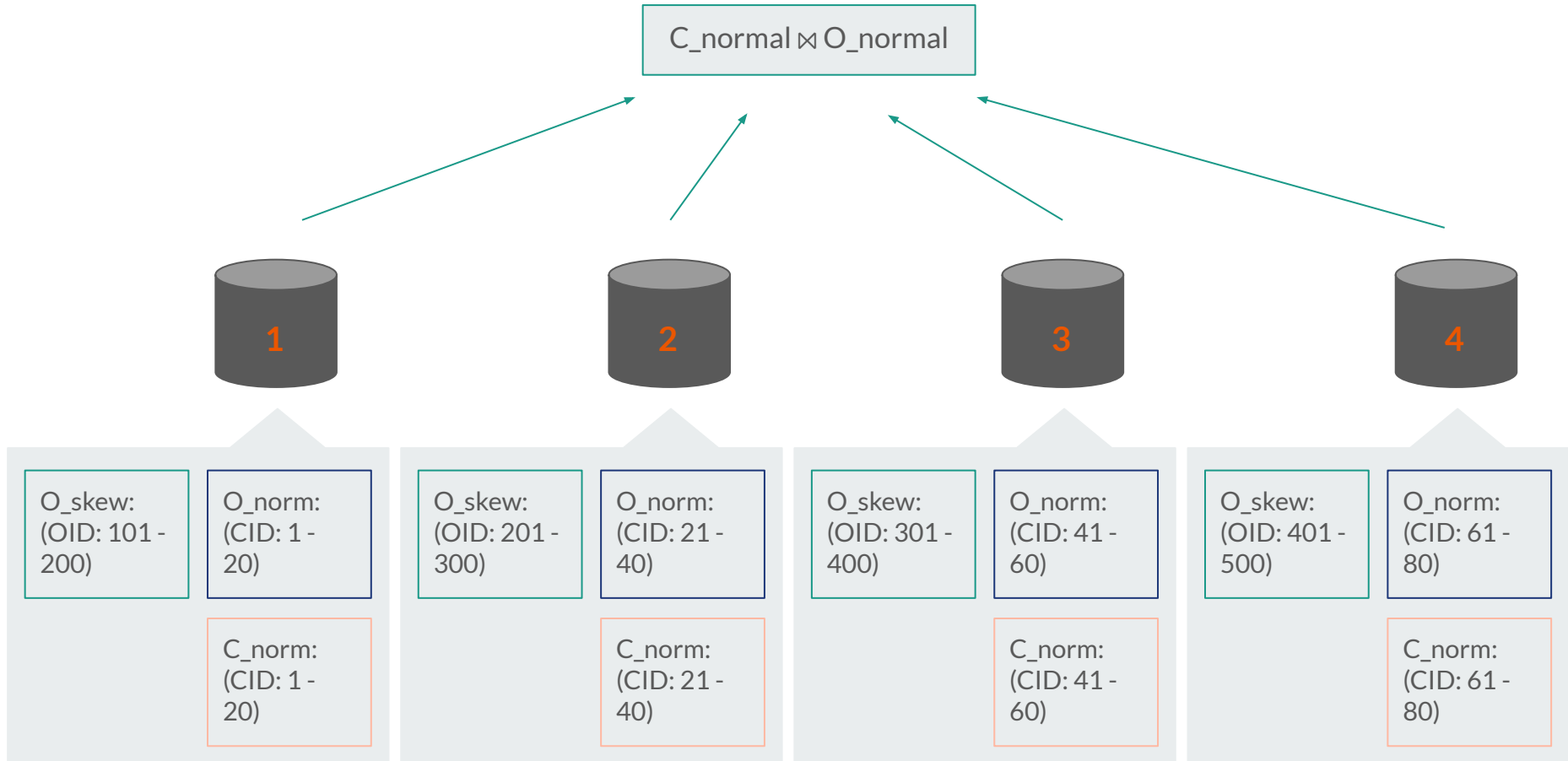
Step 2: Do a regular distributed join algorithm with normal tables. Let's use Hash/Shuffle Join.



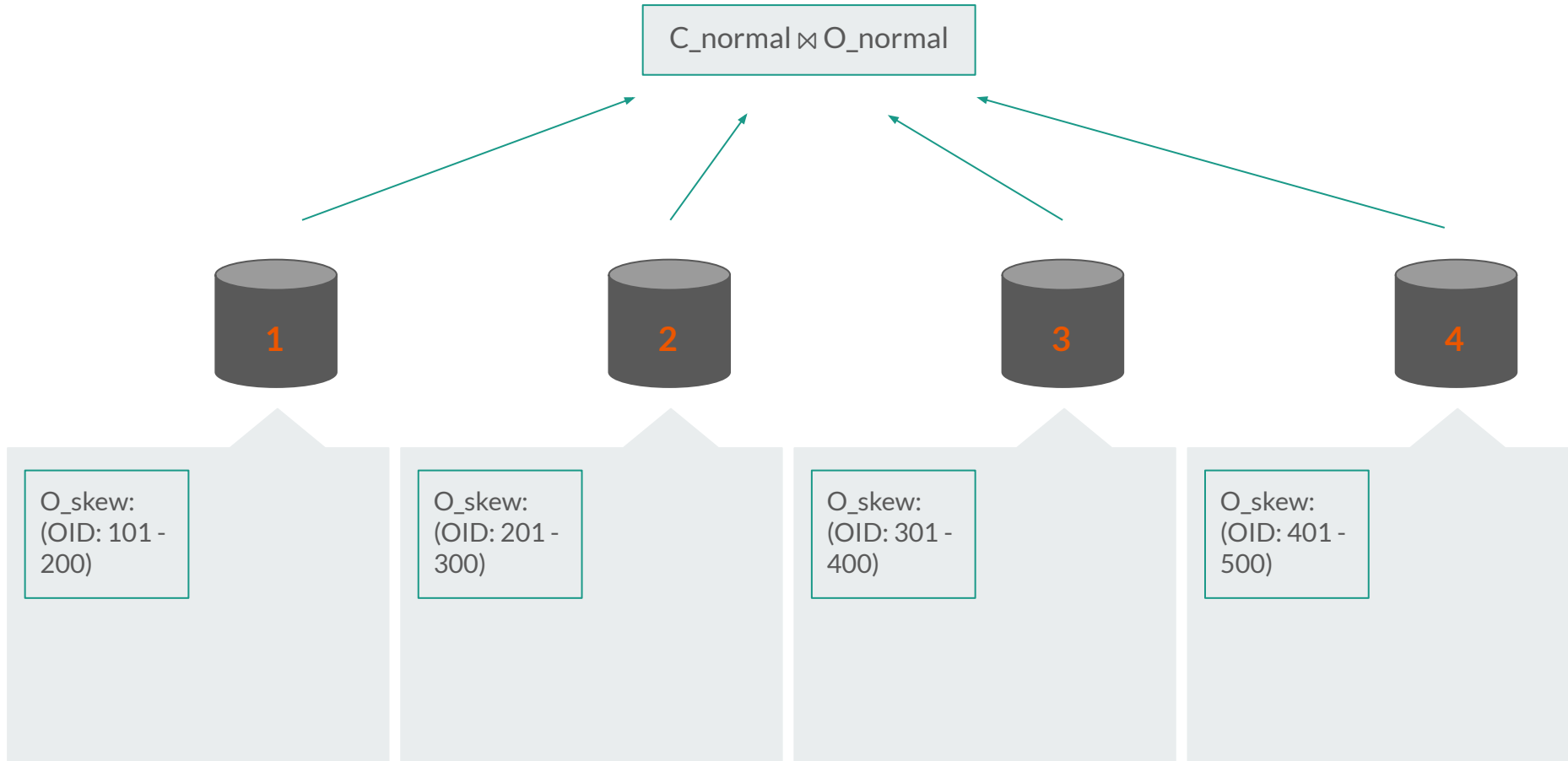
Step 2: Do a regular distributed join algorithm with normal tables. Let's use Hash Join.



Step 2: Do a regular distributed join algorithm with normal tables. Let's use Hash Join.



Step 3: Broadcast Customers_skew and perform broadcast join.



Step 3: Broadcast Customers_skew and perform broadcast join.

$C_normal \bowtie O_normal$



O_skew:
(OID: 101 -
200)

C_skew:
copy



O_skew:
(OID: 201 -
300)

C_skew:
copy



O_skew:
(OID: 301 -
400)

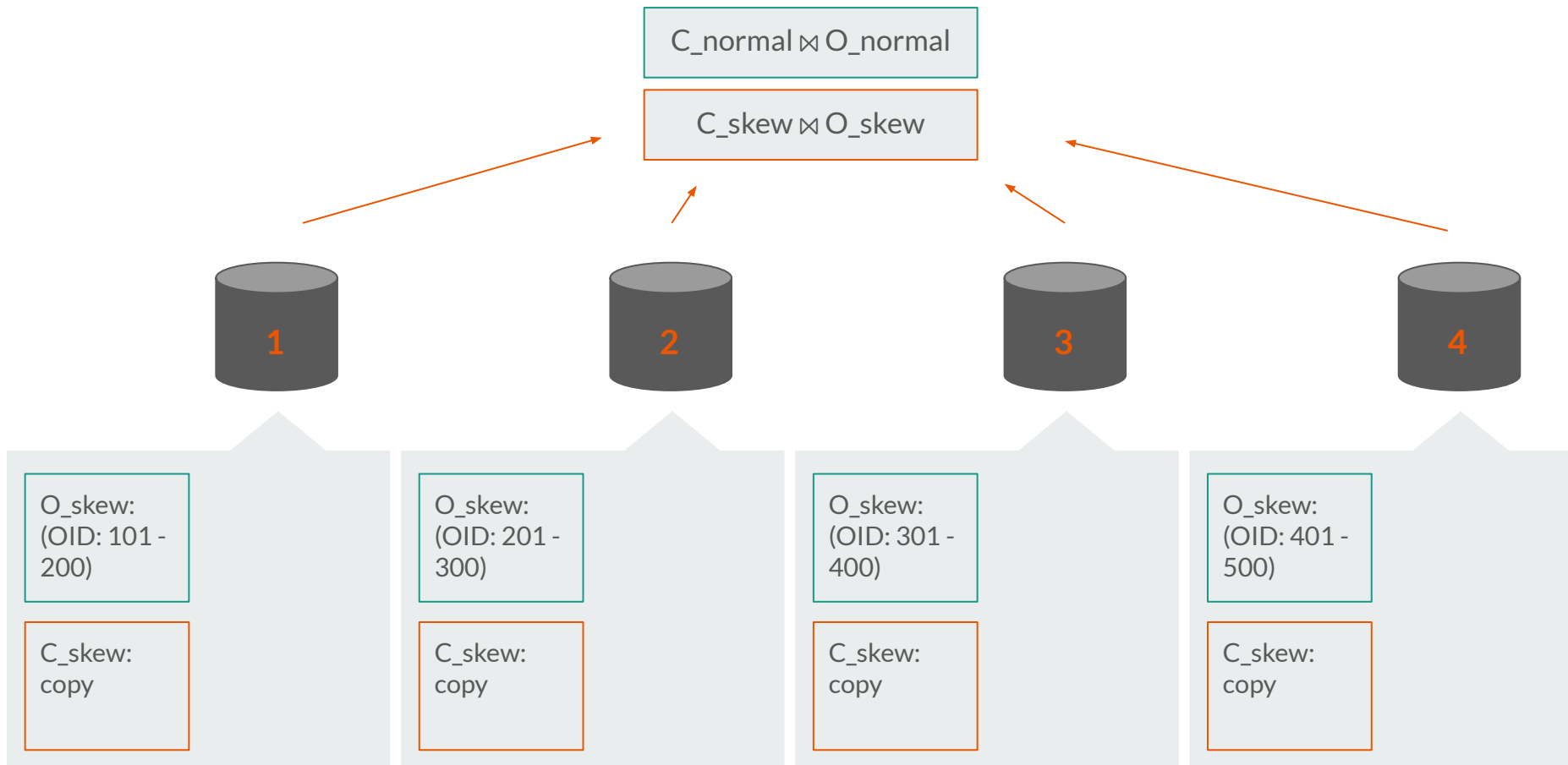
C_skew:
copy



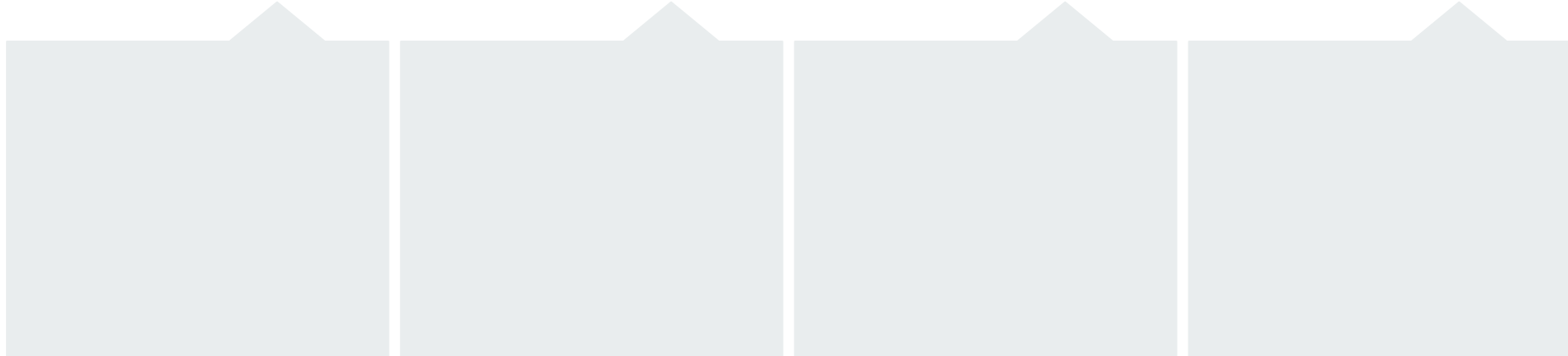
O_skew:
(OID: 401 -
500)

C_skew:
copy

Step 3: Broadcast Customers_skew and perform broadcast join.



Step 4: Union the intermediate results.



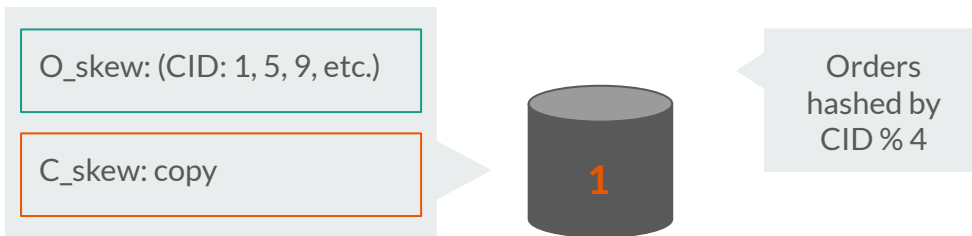


Broadcast Skew Join: Benefits

- There is limited repartitioning and data replicating / moving. We only repartition the normal tables (if we choose to do a hash join).
 - Reduced overhead
 - Reduced data movement (as opposed to regular broadcast join)
- If the initial partition was on the join key, the result of the join will retain the partitioning scheme of the non-broadcasted side.
 - No need to repartition down the line
 - E.g. GROUP BY

Broadcast Skew Join: Drawbacks

- More complicated when both sides have skew
- What is the trade-off of limited repartitioning?
 - What if our original input spreads the workload unevenly a priori? For example, what if our original partition of Orders was using the CustomerID?
 - This no longer solves our skew issue...





Full Skew Join: Setup

A scenario where there are skew key values in each table.

Tables

Customers

CID	Name	Address
1	Alice	Chicago
2	Bob	Seattle
3-26	...	...
27	Group 1	Chicago
...	Group 1	...
28	Group 2	New York
...	Group 2	...

} 24 entries for others

} 20 entries for Group 1

} 40 entries for Group 2

Orders

OID	CID	Item
101 - 180	1	...
181 - 280	2	...
281 - 282	25	...
283 - 284	26	...
285 - 324	...	...

} 80 orders for Alice

} 100 orders for Bob

} 2 orders for Group 1

} 2 orders for Group 2

} 40 orders for others



Full Skew Join

1. Identify the skewed join keys.
 - Let $S1$ be the set of high-frequency (skewed) join key values in table A.
 - Let $S2$ be the set of high-frequency (skewed) join key values in table B.
 - Let N be the remaining (normal) join key values.
2. Hash partition the rows with key values in N across the nodes.
3. For each row in A with key value in $S1$, round-robin distribute to each node.
4. For each row in B with key value in $S1$, replicate to each node.
5. For each row in B with key value in $S2$, round-robin distribute to each node.
6. For each row in A with key value in $S2$, replicate to each node.

Step 1: Split the tables

$S1 = \{1, 2\}$ (Alice and Bob)
 $S2 = \{27, 28\}$ (Group 1 and Group 2)

C_norm

CID	Name	Address
3-26	...	...

} 24 entries for others

C_skew1

CID	Name	Address
1	Alice	Chicago
2	Bob	Seattle

C_skew2

CID	Name	Address
27	Group 1	Chicago
...	Group 1	...
28	Group 2	New York
...	Group 2	...

} 20 entries for Group 1

} 40 entries for Group 2

O_norm

OID	CID	Item
285 - 324	...	...

} 40 orders for others

O_skew1

OID	CID	Item
101 - 180	1	...
181 - 280	2	...

} 80 orders for Alice
} 100 orders for Bob

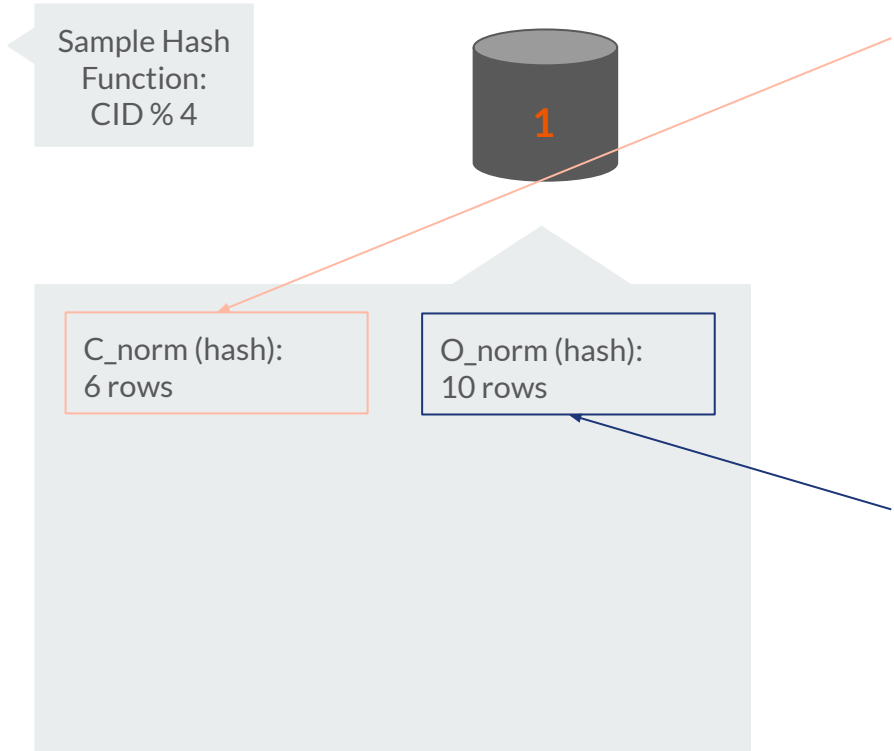
O_skew2

OID	CID	Item
281 - 282	25	...
283 - 284	26	...

} 2 orders for Group 1
} 2 orders for Group 2

Step 2: Hash rows with normal key values

S1 = {1, 2} (Alice and Bob)
S2 = {27, 28} (Group 1 and Group 2)



C_norm partition in node 1

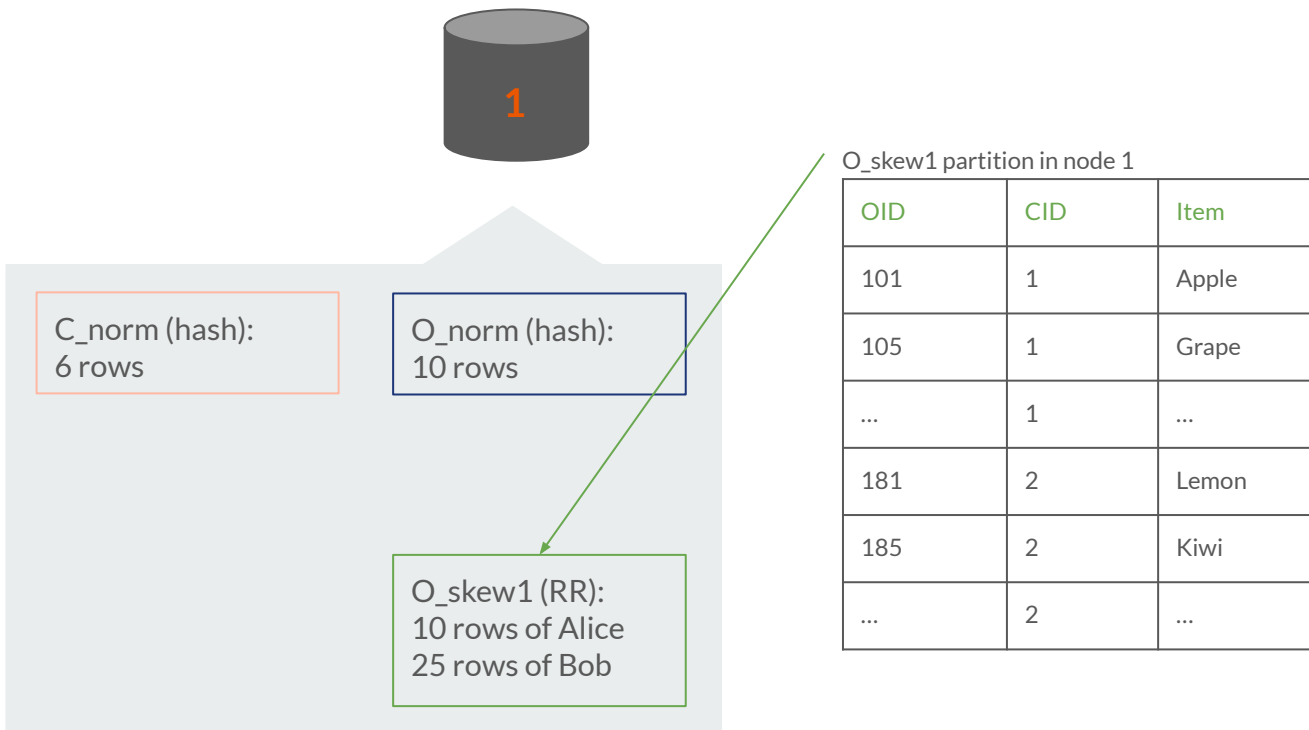
CID	Name	Address
5	Ed	Seattle
9	Ivan	Chicago
13	Max	NY
...	...	...
25	Yana	LA

O_norm partition in node 1

OID	CID	Item
287	5	Melon
291	9	Apricot
...	...	...
323	25	Cherry

Step 3: RR for S1 in Orders

S1 = {1, 2} (Alice and Bob)
S2 = {27, 28} (Group 1 and Group 2)



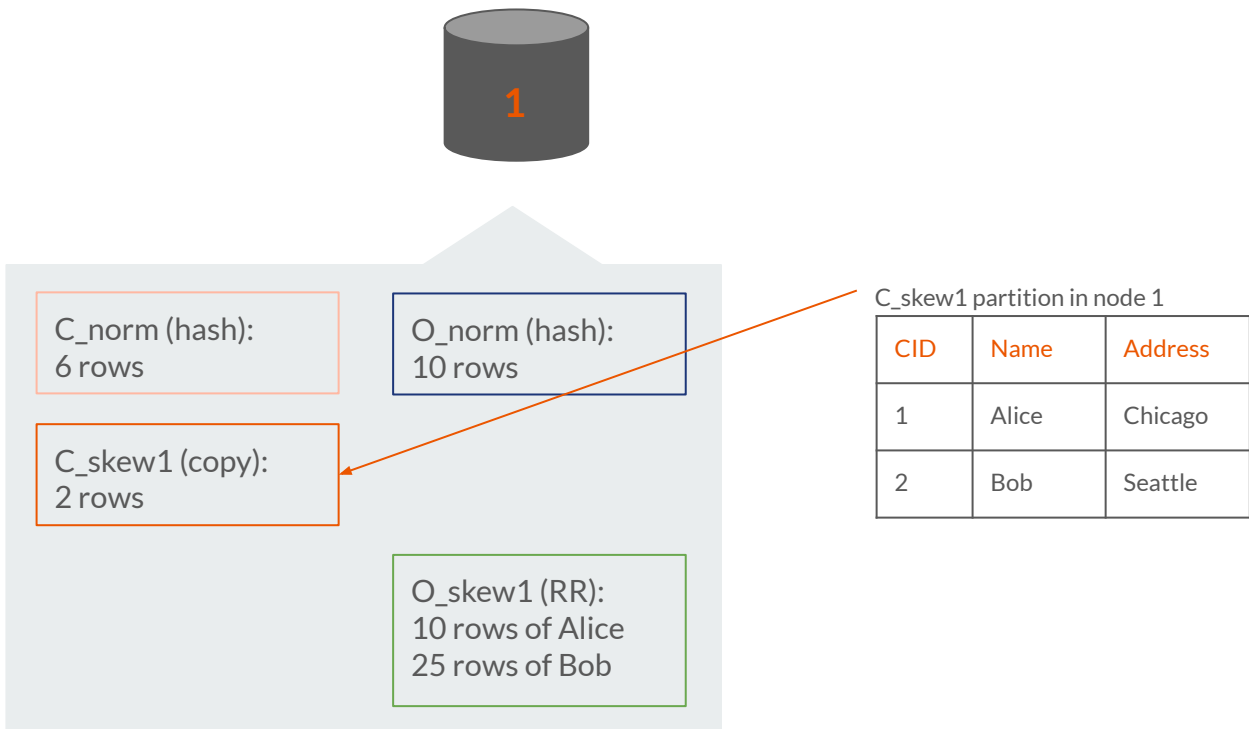
If you are a row that has a key value that's skewed in your current table, you round-robin.

If you are a row that has a key value that's skewed in the other table, you copy.

Step 4: Replicate for S1 in Customers

S1 = {1, 2} (Alice and Bob)

S2 = {27, 28} (Group 1 and Group 2)



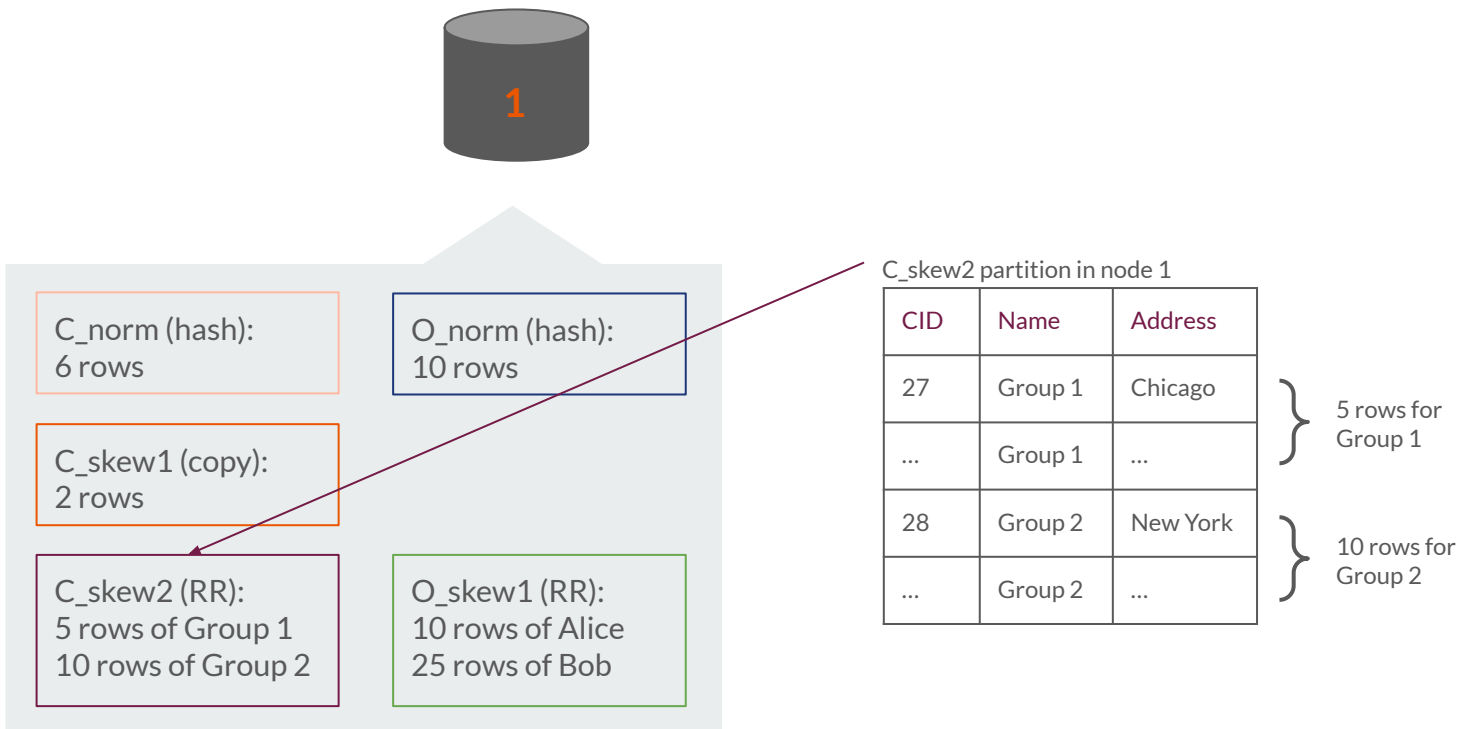
If you are a row that has a key value that's skewed in your current table, you round-robin.

If you are a row that has a key value that's skewed in the other table, you copy.

Step 5: RR for S2 in Customers

S1 = {1, 2} (Alice and Bob)

S2 = {27, 28} (Group 1 and Group 2)

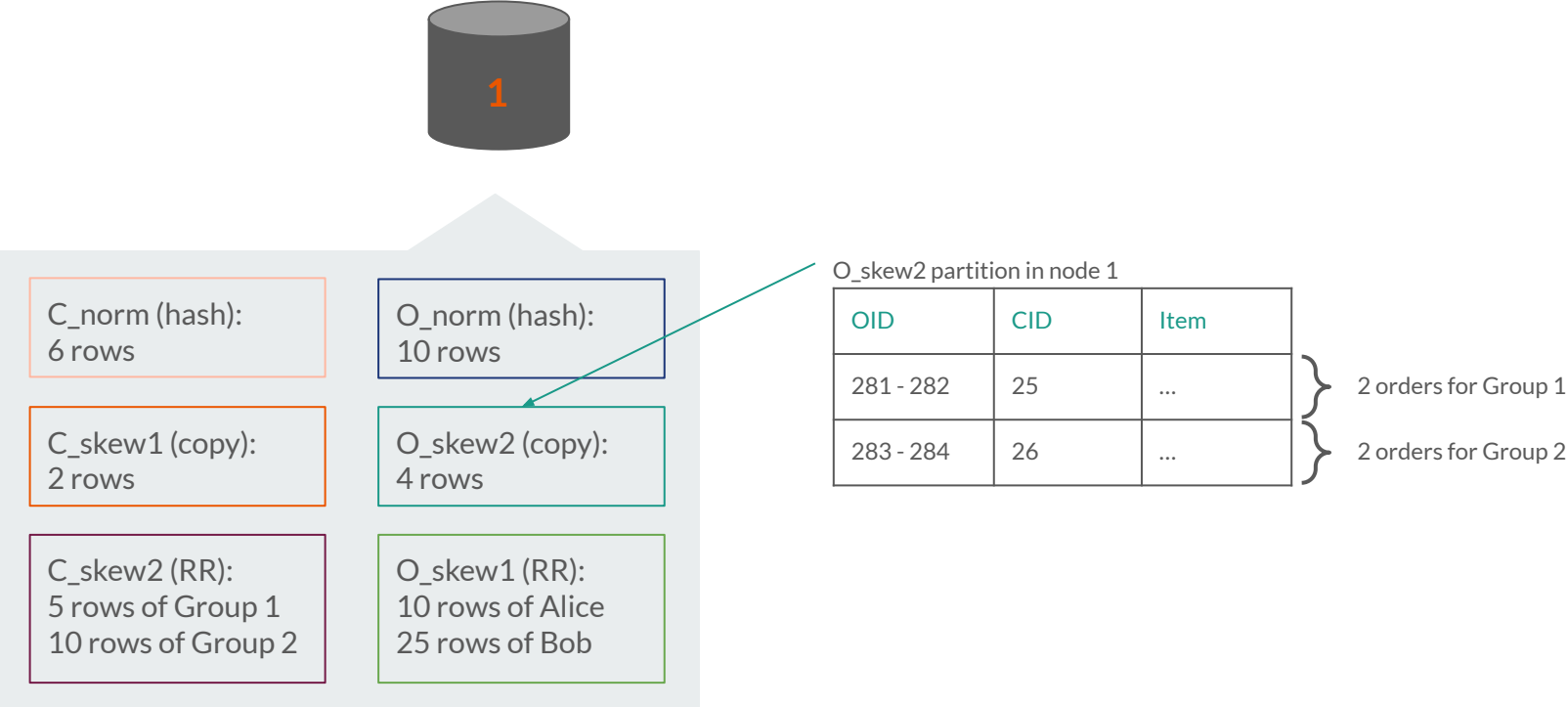


If you are a row that has a key value that's skewed in your current table, you round-robin.

If you are a row that has a key value that's skewed in the other table, you copy.

Step 6: Replicate for S2 in Orders

S1 = {1, 2} (Alice and Bob)
S2 = {27, 28} (Group 1 and Group 2)



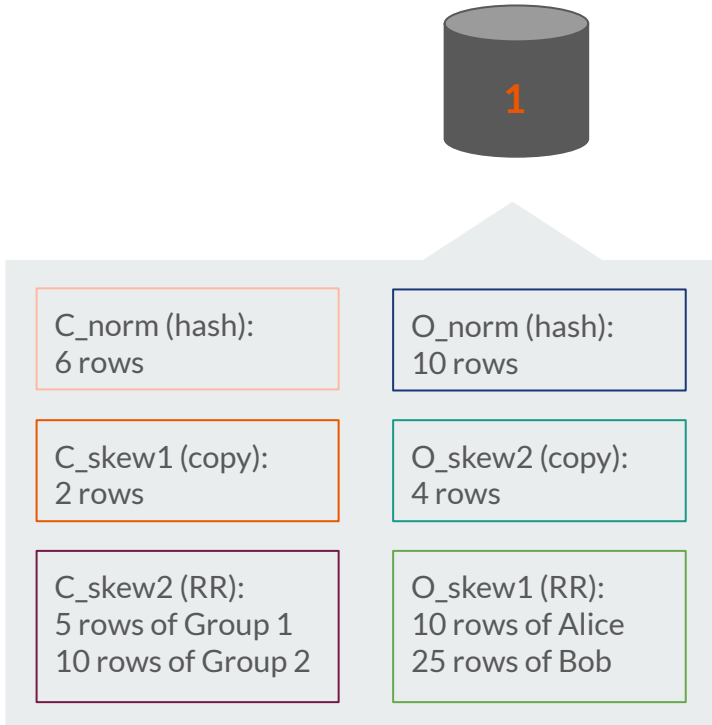
If you are a row that has a key value that's skewed in your current table, you round-robin.

If you are a row that has a key value that's skewed in the other table, you copy.

Final Partition Result

S1 = {1, 2} (Alice and Bob)

S2 = {27, 28} (Group 1 and Group 2)





Full Skew Join: Benefits

- Full skew join fully repartitions → the workload is distributed evenly.
- Since we always repartition, less sensitive to input partitions.
- Good for when both tables have skew key values. In particular, when skew key values are skew in only one table.



Full Skew Join: Drawbacks

- Repartitioning means a lot of overhead.
- We lose the existing partition; later in the execution pipeline, we may need to repartition again.



Hybrid Skew Join

Goals:

- Retain as much original partitioning as possible
- Guarantee balance of workload
- Have less overhead

Steps:

1. We use the same initial split as Broadcast Skew Join (split into normal key values and skewed key values).
2. For all normal key values, use a normal parallel join to retain partitioning.
3. For all skewed key values, use the full skew join method.
4. Union both results.

Comparison

		B-SkewJoin		F-SkewJoin		H-SkewJoin	
	Source of data skew	Left	Right	Left	Right	Left	Right
Applicability	Inner	✓	✓	✓	✓	✓	✓
	Left Outer	×	✓	✓	✓	✓	✓
	Right Outer	✓	×	✓	✓	✓	✓
	Full Outer	×	×	✓	✓	✓	✓
Partition Requirements	Repartition low-frequency keys	if necessary		Yes		if necessary	
	Repartition high-frequency keys	No		Yes		Yes	
	Require SJP	×		✓		✓	
Join Graph Topology	High-Frequency Key Source	None		Round-Robin		Round-Robin	
	High-Frequency Key Other	Broadcast		Partial Replication		Partial Replication	
	Join Low-Frequency Keys	Separate		Not Separate		Separate	
Derived Interesting Properties	Preserve Partitioning	Yes		No		Partial	
	Preserve Sort	Yes		Yes		Yes	
Cost	Overhead	Low		High		Moderate	
	Robustness	Low		High		High	
	# of Scheduled Operators	High		Low		Moderate	



References